

Оглавление

Git и GitHub	2
Теоретическая часть	2
1. Что такое Git?	2
2. Что такое GitHub?	4
3. Основные команды Git:	4
Практическая часть	5
Самостоятельная часть	8
Вопросы	8
Git. Объединение веток	9
Теоретическая часть	9
1. Что такое объединение веток и зачем оно нужно?	9
2. Как работает объединение веток?	9
3. Почему важно правильно объединять ветки?	9
4. Конфликты при слиянии	10
Практическая часть	10
Самостоятельная часть	11
Вопросы	11

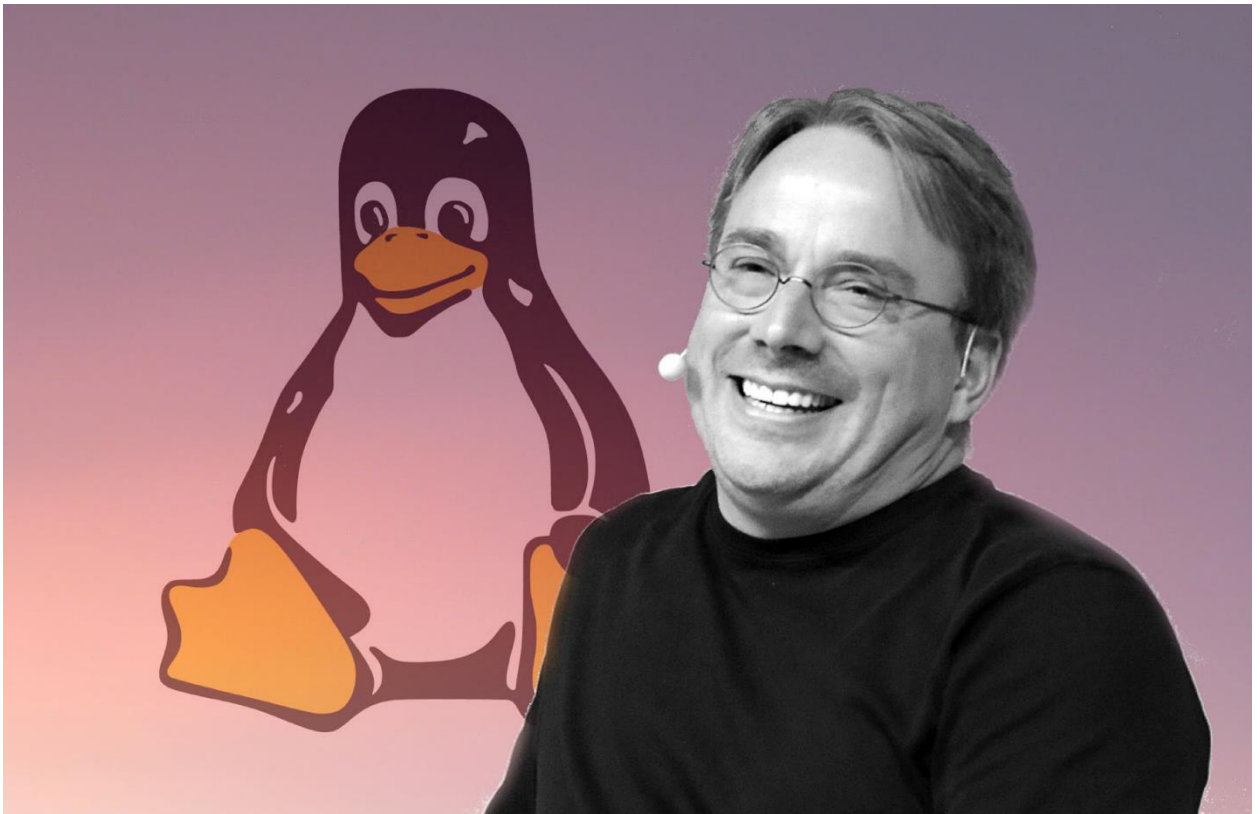
Git и GitHub

Теоретическая часть

1. Что такое Git?



Git — это распределённая система контроля версий, которая позволяет отслеживать изменения в исходном коде, работать с различными версиями файлов и сотрудничать с другими разработчиками. Git был разработан для управления большими проектами и эффективно помогает при совместной работе.



Git был создан **Линусом Торвальдсом**, финским программистом, наиболее известным как создатель операционной системы **Linux**. Он разработал Git в 2005 году, чтобы улучшить процесс управления версиями исходного кода ядра Linux.

До этого ядро Linux использовало систему контроля версий **BitKeeper**, но из-за лицензирования и ограничений на бесплатное использование возникли проблемы. Линус, не удовлетворённый существующими решениями, решил создать собственную систему контроля версий, которая бы соответствовала его нуждам — быстрая, надёжная и с поддержкой распределённой работы.

Git был спроектирован с учётом следующих ключевых требований:

- **Быстродействие:** Git должен был работать с большими проектами быстро и эффективно.
- **Распределённая модель:** Каждый разработчик должен был иметь полную копию репозитория на своём устройстве.
- **Надёжность:** Git должен был обеспечивать целостность данных, предотвращая потерю информации.

Git стал очень популярным среди разработчиков, и в настоящее время используется для управления версиями в тысячах проектов по всему миру, включая такие проекты, как Linux, Android и другие.

Основные принципы работы с Git:

- **Репозиторий (repository)** — это место, где хранятся все версии файлов. Репозиторий может быть локальным (на вашем компьютере) и удалённым (например, на GitHub).
- **Коммиты (commits)** — это снимки состояния файлов на определённый момент времени. Каждый коммит имеет уникальный идентификатор и описания изменений.
- **Ветки (branches)** — позволяют параллельно работать над разными частями проекта. Основная ветка обычно называется main или master.
- **Мерж (merge)** — процесс слияния изменений из одной ветки в другую.

2. Что такое GitHub?



GitHub — это облачный сервис для хостинга репозиториях Git. Он позволяет разработчикам хранить код, управлять проектами, отслеживать проблемы и сотрудничать с другими программистами.

Основные возможности GitHub:

- Хранение кода в репозиториях.
- Визуализация истории изменений и слияний.
- Система управления задачами (issues), запросы на слияние (pull requests).
- Возможность совместной работы и кода через форки и пулл-реквесты.
- Интеграция с различными CI/CD инструментами.

3. Основные команды Git:

- `git init` — инициализация нового репозитория.
- `git clone <url>` — клонирование репозитория.

- `git status` — проверка состояния репозитория.
- `git add <file>` — добавление изменений в область подготовки.
- `git commit -m "message"` — создание коммита с сообщением.
- `git push` — отправка изменений на удалённый сервер.
- `git pull` — получение последних изменений с удалённого репозитория.
- `git branch` — просмотр текущих веток.
- `git merge <branch>` — слияние веток.

Практическая часть

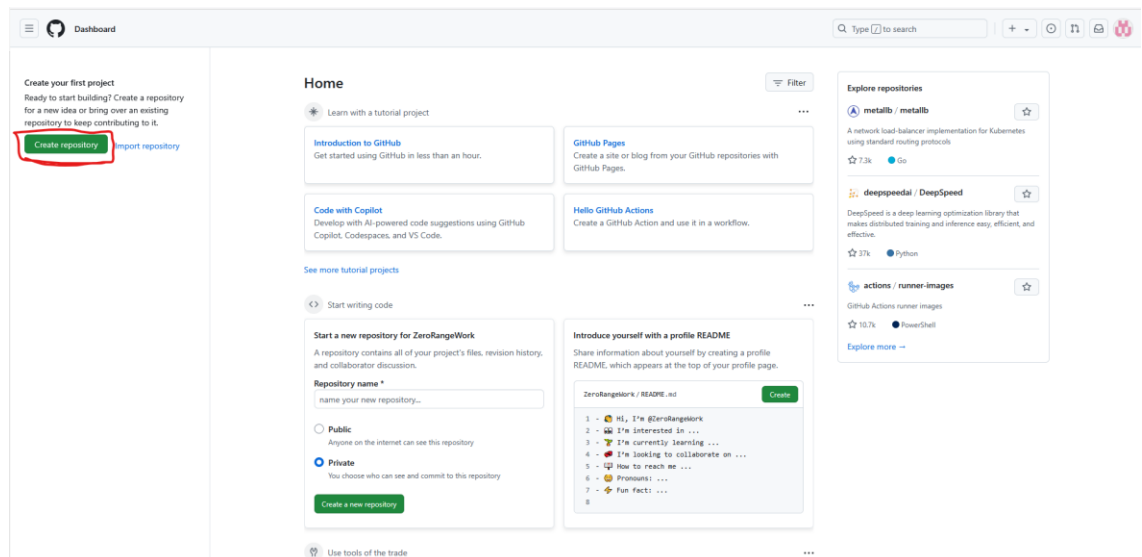
При выполнении практической часть делаем скриншоты и сохраняем их в word.

1. Проверьте установлен ли git на вашем компьютере



(если не установлен, то скачайте Git с [официального сайта](#) и следуйте инструкциям.)

2. Создайте новый репозиторий на GitHub:
3. Перейдите на [GitHub](#) и зарегистрируйтесь (или войдите).
4. Нажмите на кнопку "Create repository" для создания нового репозитория.



5. Дайте репозиторию имя, выберите его публичность (публичный или приватный) и нажмите "Create repository"

Create a new repository
A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * ZeroRangeWork / Repository name * test
test is available.

Great repository names are short and memorable. Need inspiration? How about [ideal-dollop](#)?

Description (optional)

☒ Public
Anyone on the internet can see this repository. You choose who can commit.

☐ Private
You choose who can see and commit to this repository.

Initialize this repository with:
☐ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore
.gitignore template: None
Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license
License: None
A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

① You are creating a public repository in your personal account.

Create repository

6. После создание репозитория, вас перекинет основную страницу репозитория с инструкцией

test Public

Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
[Get started with GitHub Copilot](#)

Add collaborators to this repository
Search for people using their GitHub username or email address.
[Invite collaborators](#)

Quick setup — if you've done this kind of thing before
[Set up in Desktop](#) or [HTTPS](#) [SSH](#) <https://github.com/ZeroRangeWork/test.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "e test" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ZeroRangeWork/test.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/ZeroRangeWork/test.git
git branch -M main
git push -u origin main
```

7. Затем открываем папку проекта (с помощью vs code), который хотим отправить в репозиторий.
8. Открываем терминал в vs code, и инициализируем с помощью команды:

git init

После выполнение команды, будет выводиться данный текст об успешной инициализации(также название файлы и папки могут подсвечиваться зеленым цветом):

```
PS C:\Users\ПерминовЕА\Documents\test> git init
Initialized empty Git repository in C:/Users/ПерминовЕА/Documents/test/.git/
PS C:\Users\ПерминовЕА\Documents\test>
```

(После введение команды создалась скрытая папка .git в котором храниться конфигурация репозитория)

9. Теперь подключаем локальный репозиторий (ваш компьютер) к удаленному репозиторию(github) с помощью команды:

git remote add origin ссылка на репозиторий

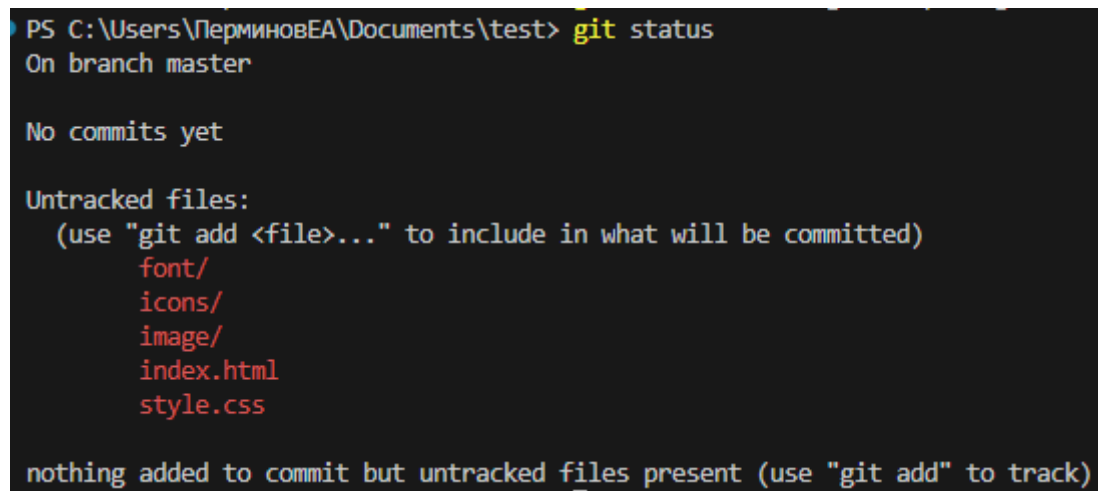
10. Создаем ветки в которых будет храниться наш проект. Обычно создают ветки prod и dev. В ветке prod храниться рабочий проект для пользователей, а в dev храниться проект в разработке.(создай для начало только ветку dev)

git branch -М название_ветки

11. Добавляем файлы в удаленный репозиторий, но для начало проверим статус файлов с помощью команды:

git status

Данная команда показывает какие файлы будут отправлены на удаленный репозиторий.



```
PS C:\Users\ПерминовЕА\Documents\test> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    font/
    icons/
    image/
    index.html
    style.css

nothing added to commit but untracked files present (use "git add" to track)
```

Чтобы изменить статус файла для отправки на удаленный репозиторий нужно использовать две команды:

git add название папки или файла

После добавление файла его статус измениться, но его нужно закоммитить чтобы отправить на удаленный репозиторий.

git commit -m “название коммита”

В коммитах обычно пишут, что было добавлено или изменено в проекте, например create project.

12. И наконец-то отправляем наш файл на удаленный репозиторий с помощью команды:

`git push origin название_ветки`

После введение данной команды в терминале будет выводиться данное сообщение:

```
PS C:\Users\ПерминовEA\Documents\test> git push origin dev
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 20 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 1011 bytes | 1011.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ZeroRangeWork/test.git
 * [new branch]      dev -> dev
```

Теперь заходим на страницу репозитория в github, и обнаруживаем наш файл.

13. Добавляем все файлы в нашем проекте с помощью команды:

`git add .`

Точка обозначает, что мы обращаемся ко всем файлом в проекте(папке).

Также придумываем коммит и пушим его.

Самостоятельная часть

1. Создайте новую папку в которой воспользуйтесь командой `git clone`. Затем добавьте или измените файл, отправьте на удаленный репозиторий

Вопросы

1. Зачем нужен `git`?
2. В чем отличия `push` и `pull`?
3. Как переместить проект из удаленного репозитория на компьютер?

Git. Объединение веток

Теоретическая часть

1. Что такое объединение веток и зачем оно нужно?

Объединение веток (merge) — это ключевой процесс в работе с системой контроля версий Git, который позволяет интегрировать изменения из одной ветки в другую. Это особенно важно при командной разработке, когда несколько человек работают над одним проектом одновременно.

Основные причины, по которым объединение веток необходимо:

- **Совместная работа:** Разработчики могут работать параллельно над различными функциями или исправлениями ошибок без влияния на основную ветку. После завершения работы изменения можно безопасно объединить.
- **Управление сложными проектами:** В больших проектах часто используются различные ветки для разных задач (например, `feature`, `bugfix`, `release`). Объединение этих веток помогает структурировать работу и упростить управление проектом.
- **Восстановление изменений:** Если какой-то код был удалён или изменён неправильно, его можно восстановить через слияние с предыдущей версией.
- **Поддержание актуальности:** Слияние позволяет обновлять ветки, чтобы они оставались синхронизированными с основной линией развития проекта.

2. Как работает объединение веток?

Git предлагает два основных способа объединения веток:

- **Fast-forward merge (прямое слияние):** Этот тип слияния происходит, когда одна ветка является "продолжением" другой. Например, если вы создали ветку `feature` от `main` и не делали коммитов в `main` после этого, Git просто переместит указатель `main` к последнему коммиту ветки `feature`. Это называется "fast-forward" слиянием, так как Git "перематывает" историю вперёд.
- **Three-way merge (трёхстороннее слияние):** Если между ветками есть разветвление (например, вы сделали изменения в обеих ветках), Git создаёт новый коммит слияния, который объединяет изменения из обеих веток. Этот процесс называется трёхсторонним слиянием, потому что Git использует три точки: общего предка (`common ancestor`), текущей ветки и ветки, которую вы хотите слить.

3. Почему важно правильно объединять ветки?

Правильное объединение веток помогает избежать проблем в будущем и поддерживать чистую историю изменений. Вот несколько причин, почему это важно:

- **Чистая история:** Принудительное создание коммитов слияния (с помощью флага `--no-ff`) позволяет видеть, какие ветки были объединены и когда. Это полезно для анализа истории проекта.
 - **Избегание конфликтов:** Регулярное слияние веток помогает минимизировать риск возникновения конфликтов, так как изменения интегрируются на ранних этапах.
 - **Стабильность проекта:** Перед слиянием веток можно протестировать их содержимое, чтобы убедиться, что изменения не приводят к ошибкам.
-

4. Конфликты при слиянии

Конфликты возникают, когда в разных ветках были внесены изменения в одну и ту же часть кода, и Git не может автоматически решить, какие изменения сохранить. Например:

- Два разработчика меняют один и тот же файл в разных местах.
- Один разработчик меняет файл, а другой удаляет его.

Практическая часть

При выполнении практической часть делаем скриншоты и сохраняем их в word.

1. Клонировать прошлый проект из репозитория или делаем в той же папки.
2. Затем создайте ветку `prod` и переключаемся к ней с помощью команды:

`git checkout название_ветки`

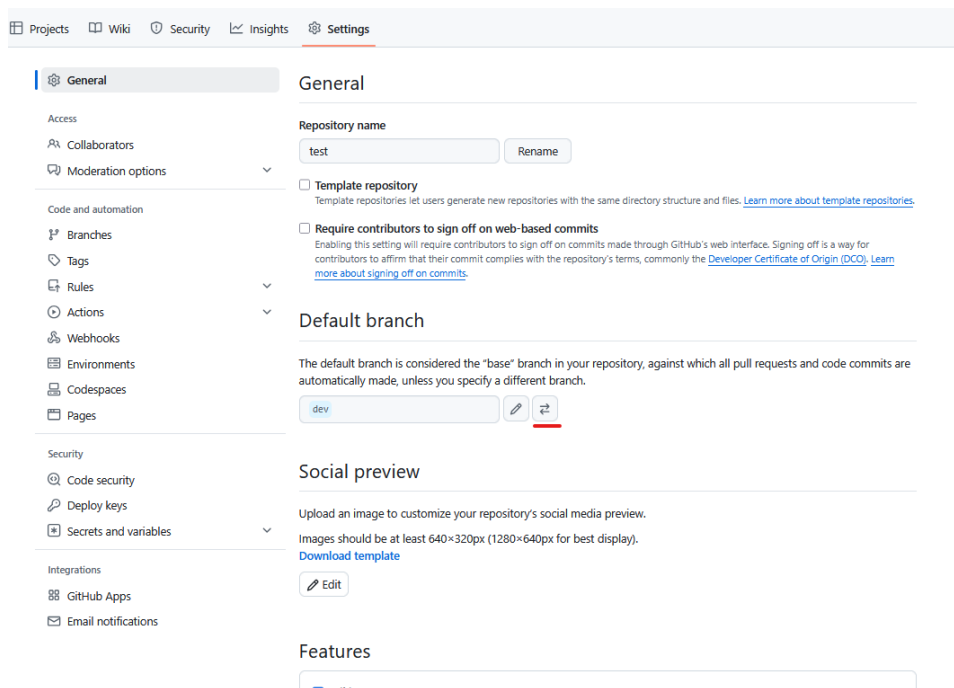
Чтобы посмотреть в какой вы ветки воспользуйтесь командой:

`git branch`

Так как ветка пустая ее нужно заполнить чем-нибудь для слияния. Для этого просто пишем:

`git push origin prod`

(зайди на страницу репозитория, перейдите в настройки и измените дефолтную ветку на `prod`. Это нужно чтобы на главной странице репозитория показывало именно, который мы указали)



3. Теперь переключаемся на ветку dev и добавляем в папку файл README.md. Также коммитим его и пушим в ветку dev.
4. Переключаемся в ветку prod и объединяемся с веткой dev:

git merge название_ветки

После введение команды высветиться данное сообщение:

```
PS C:\Users\ПерминовЕА\Documents\test> git merge dev
Updating ed6f672..a4a1924
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

5. Делаем пуш на ветку prod(коммит создал сам git)
 6. Заходим на репозиторий и наблюдаем новые файлы в ветке prod(вам также сообщит github об обновлении данной ветки)
- Данной практической использовали только прямое слияние (Fast-forward)

Самостоятельная часть

1. Создай свою ветку(сделай в ней какие либо изменения), затем сделаем объединяемся с веткой dev, а потом с веткой prod.

Вопросы

1. Какие способы объединения веток существует?
2. Какие различия в способах объединения веток, который вы перечислили в первом пункте?